

Revised Design Rationale: Golden Beetle and Interactions (Req 2)

1. Introduction

2. Design Goals

- **Implement GoldenBeetle Creature:** Define attributes (25 HP, rot immune); implement item production (GoldenBeetleEgg every 5 turns via ActorProducible interface and ProduceBehaviour); enable direct consumption by Player (Eatable, heal 15 HP, 1000 runes to Wallet); implement wandering/following AI (FollowBehaviour, egg production prioritized); allow Player attacks (AttackAction).
- **Implement GoldenBeetleEgg Mechanics:** Define as Eatable Egg; implement unique hatching (near "cursed entities" like Blight, via ItemProducible on Egg); prevent hatching in inventory; allow Player consumption from inventory (restores 20 stamina).
- **Integrate Systems:** Utilize Eatable/EatAction; leverage ActorProducible (for GoldenBeetle) and ItemProducible (for GoldenEgg hatching); incorporate Player's Wallet.
- **Promote Extensibility & Reusability:** Ensure Eatable applies to creatures/items; flexible FollowBehaviour; diverse Egg hatching conditions (though GoldenEgg is specific, the base Egg structure could support strategies); reusable capability checks.

3. Design Analysis & Decisions

3.1 Feature: Golden Beetle Egg Production Mechanism

How the GoldenBeetle (an Actor) manages its timed production of GoldenBeetleEgg (an Item). The GoldenBeetle class should implement the ActorProducible interface for this purpose.

Option A: Logic in GoldenBeetle's ActorProducible Methods and ProduceBehaviour

- **Description:** GoldenBeetle implements ActorProducible. Its canProduceOffspring() method checks an internal 5-turn timer (turnsSinceEggProduced). Its produceOffspring() method creates a new

GoldenEgg() and places it on the map. A generic ProduceBehaviour is used. This behaviour calls GoldenBeetle.canProduceOffspring(). If true, it returns a generic ProduceAction. This action, when executed, calls GoldenBeetle.produceOffspring().

- **Maintainability:** Good. The triggering mechanism is clean. Changes to the 5-turn rule or what egg is produced are localized within GoldenBeetle class.

Pros	Cons
Improved SRP: ProduceBehaviour/ProduceAction handle the generic triggering mechanism, while GoldenBeetle's ActorProducible methods encapsulate its specific rule (5-turn timer) and outcome (creating a GoldenEgg).	The ActorProducible interface being used for an Actor to produce an <i>Item</i> (the egg) is a specific interpretation of "offspring." While functional, careful naming and documentation are important.
Reusability of Behaviour/Action: ProduceBehaviour and ProduceAction are generic and can be used by any ActorProducible entity.	The 5-turn timer logic is internal to GoldenBeetle's canProduceOffspring and playTurn (for turnsSinceEggProduced++), which is localized but not easily reusable for other creatures with different timed productions without code duplication in those creatures.

Option B: Production Logic Fully Embedded in GoldenBeetle.playTurn()

- **Description:** All logic (5-turn timer, GoldenBeetleEgg creation, adding to map) would reside directly in GoldenBeetle.playTurn(). The ActorProducible interface might be trivial or unused for initiation.
- **Maintainability:** Very poor as playTurn() becomes complex and brittle.

Pros	Cons
------	------

Superficially simple if this were the only producing creature.	Severe SRP Violation: playTurn() would handle AI, movement, <i>and</i> detailed production.
	Low Cohesion: Production logic mixed with other concerns.

Decision & Justification (Option A)

- **Chosen Design:** Option A.
- **Rationale:** This design effectively uses the behaviour system (`ProduceBehaviour`, `ProduceAction`) to decouple the *initiation* of production from other AI behaviours within `GoldenBeetle.playTurn()`. The `GoldenBeetle` itself, through its implementation of `ActorProducible`, defines the specific conditions (5-turn timer, managed via `turnsSinceEggProduced`) and the outcome (creation of a `GoldenEgg`). This separation adheres well to SRP for the behaviour/action classes and keeps the production-specific rules encapsulated within the `GoldenBeetle`, fulfilling its `ActorProducible` contract. The interface `ActorProducible.java` explicitly allows for producing eggs, validating this usage.

3.2 Feature: Golden Beetle Direct Consumption by Player

The Player can consume an adjacent GoldenBeetle directly from the map, which then heals the Player and grants runes. The `GoldenBeetle.java` implements `Eatable`.

Option A: GoldenBeetle Implements Eatable, EatAction Handles Target

- **Description:** `GoldenBeetle` implements the `Eatable` interface. Its `eatenBy()` method defines the effects (heal Player +15 HP, add 1000 runes to Player's Wallet, remove self from map). Its `allowableActions()` method adds new `EatAction(this)`. The generic `EatAction` (from `EatAction.java`) takes an `Eatable` target and calls its `eatenBy()` method.
- **Maintainability:** Good. Effects are tied to the `Eatable` creature. The action remains generic.

Pros	Cons
Reuses Eatable Interface (ISP, DIP): Consistent capability definition. EatAction depends on the Eatable abstraction.	EatAction is simple and relies entirely on the Eatable target's eatenBy method for all logic, which is appropriate here.
Good OCP: Other creatures can be made directly consumable by implementing Eatable and defining their eatenBy() effects, without changing EatAction.	

Option B: Specific ConsumeGoldenBeetleAction without Eatable Interface on Creature

- **Description:** A new, highly specific ConsumeGoldenBeetleAction would be created, only working if the target is instanceof GoldenBeetle. It would directly call unique methods on GoldenBeetle. GoldenBeetle would not implement Eatable.

Pros	Cons
Action logic might seem very direct for this one case.	Violates OCP & DRY; Violates DIP; Scalability Issues; Poor maintainability.

• Decision & Justification (Option A)

- **Chosen Design:** Option A.
- **Rationale:** This design, where GoldenBeetle implements Eatable, is superior due to its adherence to DIP and OCP, and accurately reflects the provided code. By using the Eatable interface, the EatAction polymorphically handles any Eatable entity. This makes the system extensible to other directly consumable creatures. The specific consequences of being eaten are well-encapsulated within GoldenBeetle.eatenBy().

3.3 Feature: Golden Beetle Following Player

GoldenBeetle can wander and follow the Player using FollowBehaviour.

GoldenBeetle.java adds FollowBehaviour to its behaviours.

Option A: FollowBehaviour

- **Description:** The GoldenBeetle class has a FollowBehaviour instance added to its list of behaviours. The Creature base class's playTurn method iterates through behaviours, and if FollowBehaviour.getAction() returns a movement action towards a followable target (like the Player), it's executed.
- **Maintainability:** Good. Following logic is isolated.

Pros	Cons
SRP & High Cohesion: FollowBehaviour encapsulates all logic for following.	FollowBehaviour needs map access and target identification logic (which it has, checking for GeneralCapability.FOLLOWABLE).
Reusability (DRY): FollowBehaviour can be reused.	

Option B: Following Logic Embedded Directly in GoldenBeetle.playTurn()

- **Description:** All logic for detecting and moving towards the Player written directly in GoldenBeetle.playTurn().

Pros	Cons
Might seem simpler for a single creature.	SRP Violation; Low Cohesion, Poor DRY; Poor OCP
Reusability (DRY): FollowBehaviour can be reused.	

Decision & Justification

- **Chosen Design:** Option A.
- **Rationale:** Using a dedicated FollowBehaviour (Strategy Pattern), as implemented in FollowBehaviour.java and used by GoldenBeetle.java, is a strong design. It adheres to SRP, is reusable (DRY), and supports OCP for different strategies, improving maintainability over embedding AI logic directly (Option B).

3.4 Feature: GoldenBeetleEgg Hatching Logic (Conditional - near cursed entities)

GoldenBeetleEgg hatches into a GoldenBeetle only when near "cursed entities" (e.g., Blight). GoldenEgg.java extends Egg (which implements ItemProducible).

Option A: Logic Embedded Directly in GoldenEgg.canproduce() and produce()

- **Description:** GoldenEgg.java overrides canproduce(Location) and produce(Location, GameMap) from its Egg superclass (which implements ItemProducible). The GoldenEgg.canproduce() method contains the specific logic to scan its current location and adjacent tiles for ground or other entities with CURSED. If found, produce() creates a new GoldenBeetle(). The Egg.tick() method drives this by calling these overridden methods.
- **Maintainability:** Good for *this specific, unique* condition. If many eggs had different, complex environmental checks, a strategy pattern (HatchingCondition objects) would be more maintainable across all egg types.

Pros	Cons
Direct Implementation of Specific Rule: The unique hatching rule for GoldenEgg is clearly encapsulated within its own class.	Less Flexible than Strategy Pattern for <i>this specific egg</i>: Unlike SheepEgg which used a list of HatchingCondition strategies, GoldenEgg hardcodes its cursed environment check. If GoldenEgg needed <i>multiple alternative</i> ways to hatch, this would be less OCP-compliant than the strategy list.
Leverages Egg.tick() and ItemProducible Framework: Uses the	

existing mechanism for items on the ground to be updated each turn.	
---	--

Option B: GoldenEgg Uses a Specific CursedEnvironmentHatchCondition Strategy Object

- **Description:** Similar to SheepEgg from REQ1, GoldenEgg would hold a list of HatchingCondition objects, one of which would be an instance of CursedEnvironmentHatchCondition.java. The GoldenEgg.canproduce() would iterate this list and delegate the check to the strategy.
- **Maintainability:** Very high for managing diverse and evolving hatching rules.

Pros	Cons
Excellent Extensibility & OCP for Hatching Rules: New ways for GoldenEgg to hatch (or other eggs) could be added as new HatchingCondition strategies without modifying GoldenEgg's core logic.	Slightly more indirection than directly coding the check in GoldenEgg.canproduce().
High Cohesion within HatchingCondition: The "cursed proximity" logic is encapsulated in CursedEnvironmentHatchCondition.	

Decision & Justification

- o **Chosen Design:** Option A.
- o **Rationale:** This decision reflects the actual implementation in GoldenEgg. The code shows the logic for checking cursed environments directly within GoldenEgg.canproduce(), rather than instantiating and using CursedEnvironmentHatchCondition as a strategy object in a list. While Option B (using the HatchCondition strategy pattern as SheepEgg does) would offer better OCP and flexibility if the GoldenEgg itself needed multiple, interchangeable hatching conditions, the current implementation in

GoldenEgg is a direct and specific solution for its unique "cursed environment" requirement. It leverages the ItemProducible contract from its Egg parent and localizes its specific condition check. This is a simpler approach for a single, fixed condition for this particular egg type. The CursedEnvironmentHatchCondition exists and correctly encapsulates the logic, but GoldenEgg re-implements this logic internally instead of using that class as a strategy.

3.5 Feature: GoldenBeetleEgg Consumption from Inventory (Stamina Restore)

Player consumes GoldenBeetleEgg from inventory to restore 20 stamina.

GoldenEgg.java extends Egg and implements Eatable.

Option A: Generic EatAction using Eatable Interface on GoldenEgg (Actual Implemented Approach)

- **Description:** GoldenEgg implements Eatable (inherited from Egg and then specifically implemented). Its eatenBy() method defines the stamina restoration effect (restores 20 stamina, checks for BaseActorAttributes.STAMINA). The generic EatAction (from EatAction.java) handles the consumption.
- **Maintainability:** Very high. New egg effects confined to the egg's eatenBy().

Pros	Cons
Highly Reusable (DRY), Excellent OCP, Low Coupling (DIP)	Eatable interface's eatenBy() must be flexible for various effects (which it is).
High Cohesion for EatAction; Consistent pattern.	

Option B: Specific EatGoldenEggAction

- **Description:** A new action class dedicated to eating GoldenEggs.

Pros	Cons
------	------

Action is very specific.	Violates DRY, OCP; High coupling; Proliferation of classes.
--------------------------	---

Decision & Justification

- **Chosen Design:** Option A.
- **Rationale:** This is identical to the reasoning for SheepEgg consumption in REQ1 and reflects the code in GoldenEgg.java and EatAction.java. The generic EatAction with the Eatable interface is robust, maintainable, and extensible.

4. Overall Design Summary for Requirement 2

- **Golden Beetle Egg Production:** GoldenBeetle implements ActorProducible, production initiated by ProduceBehaviour/Action, with 5-turn timer logic in its interface methods.
- **Golden Beetle Direct Consumption:** GoldenBeetle implements Eatable; a generic EatAction handles consumption.
- **Golden Beetle Following AI:** Uses a dedicated, reusable FollowBehaviour.
- **GoldenBeetleEgg Hatching:** Uses ItemProducible (on Egg base), with "cursed environment" check logic directly in GoldenEgg.canproduce().
- **GoldenBeetleEgg Consumption:** Uses the generic Eatable interface and EatAction.

5. Conclusion

This design for Requirement 2 effectively integrates the Golden Beetle by leveraging established interfaces (Eatable, ActorProducible, ItemProducible) and behaviour patterns (ProduceBehaviour, FollowBehaviour). The chosen approaches, reflecting the provided Java code, prioritize SRP, OCP, DIP, and DRY where appropriate, leading to a maintainable system aligned with the REQ2 UML. Direct consumption of creatures and item production by actors are handled via interfaces, while specific AI and hatching conditions are encapsulated effectively.